Consumo de una API con API Key (curl, Postman y Python)

1. Título

"Elige tu API: autenticación con API Key en tres herramientas"

2. Objetivo general

Que el estudiante consuma una API REST protegida con API Key, elegida por él, usando línea de comandos (curl), un cliente gráfico (Postman) y código (Python), y que compare las tres herramientas y proteja su credencial.

3. Objetivos específicos

- Explicar qué es una API Key y las formas de enviarla (parámetro en la URL o header).
- Construir peticiones GET con parámetros y credencial en curl, Postman y Python.
- Interpretar los códigos de estado HTTP (200, 401, 403, 404, 429).
- Manejar errores y *timeouts* en un script.
- Proteger la clave con variables de entorno y archivos excluidos del control de versiones.

4. Duración y nivel

2 a 2.5 horas, nivel básico-intermedio. De forma individual

5. Requisitos previos

- Conocimientos básicos de HTTP y JSON.
- Python 3 con pip, Postman (o versión web) y terminal con curl.
- Un correo electrónico para registrarse en el proveedor elegido.

6. Selección de la API

Cada estudiante elige **una** de las siguientes opciones y la registra en su documento de entrega:

#	API	Qué consulta	Envío de la clave	Registro
1	OpenWeatherMap	Clima actual de una ciudad	Parámetro appid	openweathermap.org

#	API	Qué consulta	Envío de la clave	Registro
2	WeatherAPI.com	Clima actual y pronóstico	Parámetro key	weatherapi.com
3	NASA Open APIs	Foto astronómica del día	Parámetro api_key (existe DEMO_KEY para pruebas)	api.nasa.gov
4	NewsAPI	Noticias por país o tema	Header X-Api-Key (también admite parámetro)	newsapi.org
5	OMDb API	Datos de una película por título	Parámetro apikey	omdbapi.com
6	The Cat API	Imágenes y razas de gatos	Header x-api-key	thecatapi.com

Ejemplos de endpoint (verificar en la documentación oficial):

- OpenWeatherMap: https://api.openweathermap.org/data/2.5/weather con q, units y lang.
- WeatherAPI.com: https://api.weatherapi.com/v1/current.json con q y lang.
- NASA: https://api.nasa.gov/planetary/apod con date opcional.
- NewsAPI: https://newsapi.org/v2/top-headlines con country o q.
- OMDb: https://www.omdbapi.com/ con t (título) o i (identificador).
- The Cat API: https://api.thecatapi.com/v1/images/search con limit.

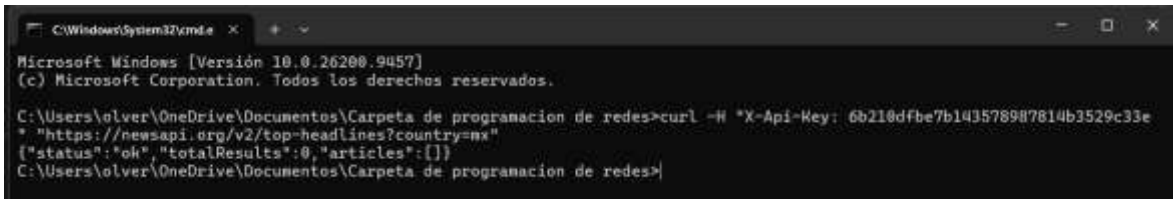
7. Desarrollo de la actividad

Fase 1. Preparación (20 min)

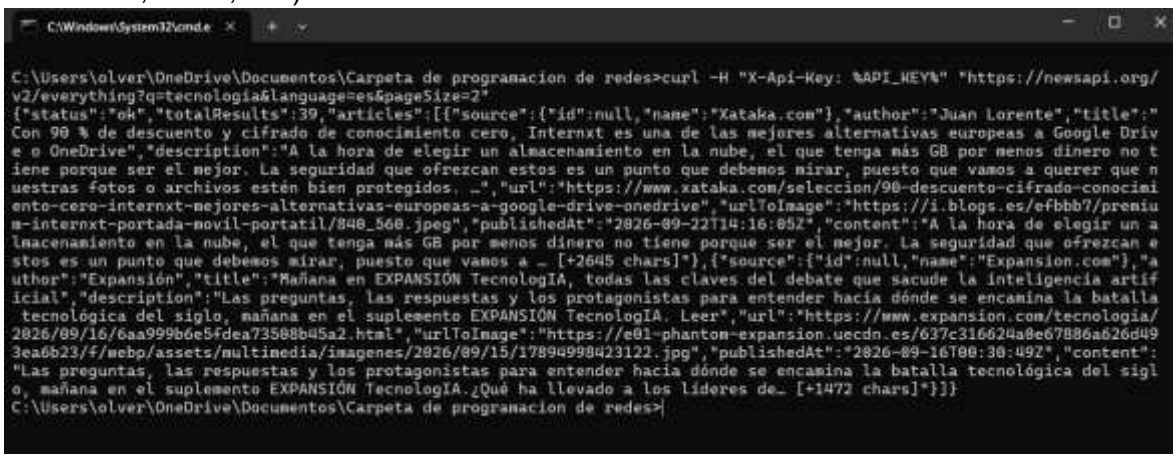
1. Elegir la API y registrarse para obtener la clave.
2. Leer la documentación del endpoint elegido y completar esta **ficha técnica**:
    - URL base y endpoint. ○ Método HTTP. ○ Parámetros obligatorios y opcionales. ○ Forma de enviar la clave (URL o header) y nombre exacto del parámetro o header. ○ Límite de uso del plan gratuito.
    - Campos de la respuesta JSON que se usarán.
3. Responder por escrito: ¿qué riesgos tiene enviar la clave en la URL frente a enviarla en un header?
4. Esperar la activación de la clave si el proveedor lo indica. Algunos tardan varios minutos y mientras tanto devuelven 401.

Fase 2. Consumo con curl (25 min)

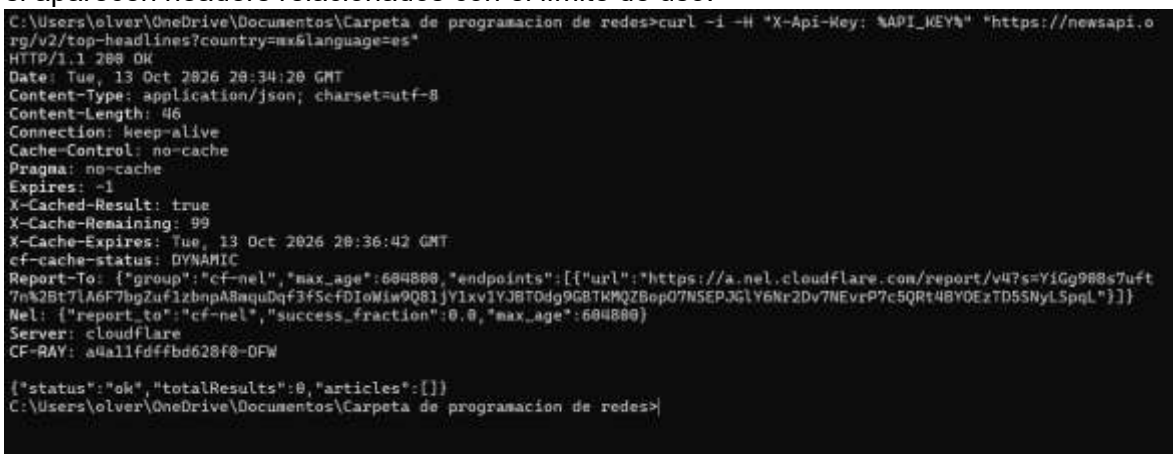
1. Hacer una petición básica con todos los parámetros obligatorios y la clave escrita directamente.



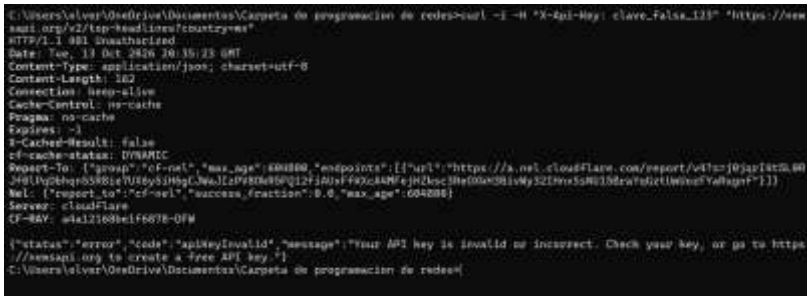
2. Repetir la petición guardando la clave en una variable de entorno (export API_KEY=...) y usándola en el comando.
3. Agregar al menos un parámetro opcional (idioma, unidades, cantidad de resultados, fecha, etc.).



4. Usar -i o -v para observar el código de estado y los headers de respuesta. Anotar si aparecen headers relacionados con el límite de uso.



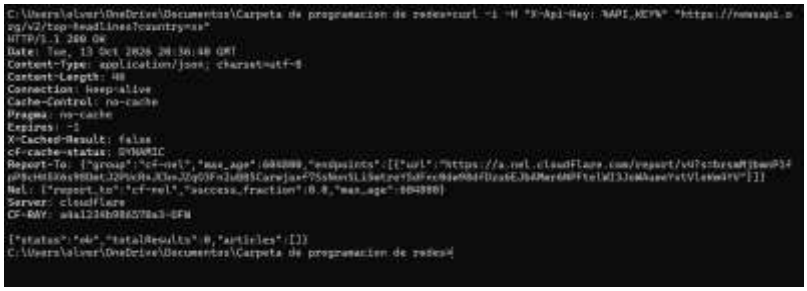
5. Provocar errores intencionales y registrar el código y el mensaje de cada uno:
    - Clave inválida.



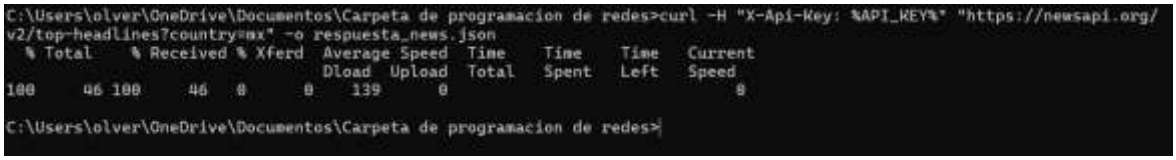
    - Parámetro obligatorio omitido.



    - Valor inexistente (ciudad, título de película, fecha fuera de rango).

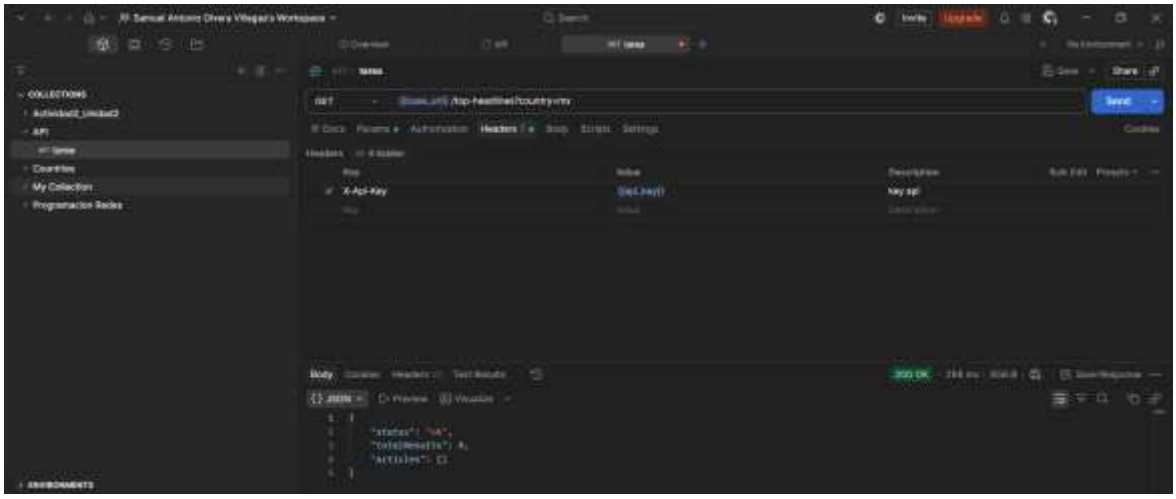


6. Guardar una respuesta exitosa en un archivo .json.



Fase 3. Consumo con Postman (30 min)

1. Crear una colección con el nombre de la API elegida.
2. Crear un entorno con las variables base_url y api_key.
3. Crear una petición GET usando las variables. Si la clave va en parámetro, ponerla en la pestaña *Params*. Si va en header, ponerla en *Headers* (o en la pestaña *Authorization*, tipo API Key).



4. Enviar la petición y analizar Body, Headers y Status.
5. En la pestaña *Tests*, escribir al menos dos verificaciones: código de estado 200 y presencia de un campo clave de la respuesta (el que se anotó en la ficha técnica).
6. Duplicar la petición cambiando un parámetro y comparar resultados.
7. Crear una tercera petición con la clave inválida y verificar con un test que el código de estado sea el de error esperado.



8. Exportar la colección **sin la clave real**.

Fase 4. Consumo con Python (40 min)

1. Crear un entorno virtual e instalar requests (opcionalmente python-dotenv).
2. Guardar la clave en un archivo .env, agregarlo a .gitignore y crear un .env.example con un valor ficticio.
3. Escribir un script que:
   - Lea la clave desde una variable de entorno y falle con un mensaje claro si no existe. ○ Reciba por teclado o argumento el valor principal de la consulta (ciudad, título, país, etc.). ○ Use requests.get() con params y, si corresponde, headers. ○ Defina un timeout. ○ Maneje Timeout, ConnectionError y respuestas no exitosas (401, 404, 429), mostrando un mensaje distinto para cada una.
   - Extraiga e imprima al menos cuatro campos relevantes de la respuesta, formateados de manera legible.

4. Extensión opcional: consultar varios valores en un ciclo, respetar una pausa entre peticiones y guardar los resultados en un archivo CSV o JSON.

Fase 5. Cierre y comparación (20 min)

1. Completar una tabla comparativa de curl, Postman y Python con estos criterios: facilidad de uso, automatización, depuración, reutilización y seguridad de la clave.
2. Puesta en común: los estudiantes que usaron APIs distintas comparan cómo se envía la clave y qué dificultades encontraron.

8. Buenas prácticas a reforzar

- No subir claves a repositorios ni compartirlas en capturas de pantalla.
- Usar variables de entorno o gestores de secretos.
- Definir siempre un timeout.
- Respetar los límites del plan gratuito y no ejecutar ciclos sin pausa.
- Regenerar la clave si se expone.
- Revisar los términos de uso de cada API. Algunos planes gratuitos restringen el uso a desarrollo o a fines no comerciales.

9. Entregables

1. **Ficha técnica** de la API elegida (Fase 1).
2. Registro de los comandos curl con sus respuestas, incluidos los errores provocados.
3. Colección de Postman exportada, con la clave oculta.
4. Script de Python funcional y archivo .env.example.
5. Documento de una página con la tabla comparativa y las respuestas de reflexión.

10. Criterios de evaluación

Criterio	Ponderación
Ficha técnica completa y correcta	15%
Peticiones curl correctas y documentadas (incluye errores)	25%
Colección Postman con variables y tests	25%
Script Python funcional con manejo de errores	35%
Total	**100%**

11. Preguntas de reflexión

1. ¿Qué diferencia hay entre autenticarse con API Key y con OAuth o JWT?

2. ¿Qué ventajas y riesgos tiene enviar la clave en la URL frente a un header?
3. ¿En qué situación elegirías curl, Postman o Python?
4. ¿Qué pasaría si tu clave aparece en un repositorio público? ¿Qué harías?
5. ¿Cómo manejarías el límite de peticiones de tu API en un script que corre todos los días?